

FASE 3 — Conversión (aplicación → persona + oportunidad + propuesta)

Objetivo: convertir el clic en “**Aplicar a beca**” del explorador en una cadena completa y **transaccional** de registros del CRM, respetando el principio **consent-first** y el **modelo de negocio** de la oferta. Idempotente ante doble clic / reintentos.

1. RPC transaccional `fn_convertir_aplicacion(jsonb)`

Migración `20260829120500_leadcenter_fn_convertir_aplicacion.sql` . SECURITY DEFINER , search_path=public , todo dentro de una sola transacción de función (atómica). Devuelve `jsonb` .

Pasos que ejecuta:

1. **Idempotencia**: si ya existe una oportunidad con esa `clave_idempotencia` , la devuelve sin crear nada nuevo.
2. **Oferta**: lee `ofertas_academicas` → `universidad_id` , `programa_id` , `sede_id` , `modelo_negocio` (fallback `por_inscrito`).
3. **Persona**: reutiliza por `celular_e164` (o por `visitante_id`); si no existe, la crea con `estado_relacion='lead'` , `telefono_verificado=true` , `metodo_verificacion='simulated'` . Nunca sobrescribe datos existentes con NULL.
4. **Consentimientos**: inserta en `consentimientos_persona` cada tipo recibido con su `estado` (otorgado / pendiente), flags `autoriza_*` , `version_texto` , `ip_origen` . El de `transferencia_universidad` guarda además `universidad_id` .
5. **Oportunidad**: la crea en la **etapa inicial** (primera no-final por `orden`) con `modelo_negocio_snapshot` y `clave_idempotencia` ; registra `historial_etapas_oportunidad` .
6. **Aplicación** (`aplicaciones` , estado `enviada`).
7. **Propuesta** (`propuestas_comerciales`) + **versión snapshot inmutable** (`versiones_propuesta_comercial`) con `oferta_snapshot` (jsonb) y la advertencia fija “Esta propuesta es informativa. BuscoEdu no garantiza admisión.”.
8. **Lógica por modelo**:
 - **por_inscrito** → asigna asesor activo (el de menor carga; **fallback a super_admin**) y registra `asignaciones_oportunidad` (tipo `automatica`).
 - **por_lead** → si hay consentimiento de **transferencia_universidad otorgado**, crea `transferencias_universidad` en estado **pendiente** con `consentimiento_id` (NOT NULL). Si **falta**, NO crea transferencia y marca la oportunidad como pendiente de consentimiento en `notas_internas` .
9. **Evento de negocio** (`eventos_negocio` , tipo `aplicacion_beca`).
10. Devuelve ids + `modelo_negocio` + `requiere_consentimiento_transferencia` .

Ante cualquier excepción, retorna `{ok:false, error:'excepcion', detalle}` y la transacción de función revierte los cambios.

2. API POST `/api/leadcenter/convertir`

- Normaliza el celular (E.164) y valida `oferta` + `clave_idempotencia` .
- **Verifica titularidad**: exige un `desafios_otp` en estado `verificado` en los últimos 15 min para ese celular (si no, `403 celular_no_verificado`). Así la conversión no puede saltarse el OTP.
- Llama al RPC con service role y devuelve su resultado.

3. API GET /api/leadcenter/consentimientos

Devuelve los `tipos_consentimiento` activos (código, nombre, texto, obligatorio) para pintarlos en el flujo **sin casillas preseleccionadas**.

4. UI — flujo consent-first

- `components/leadcenter/AplicacionConsentimientoModal.tsx` : `datos` → `OTP` → `consentimientos` → `conversión` → `confirmación` . Ninguna casilla viene marcada; el consentimiento obligatorio (tratamiento de datos) se valida antes de enviar. Si el modelo es `por_lead` , explica que se requiere autorizar la transferencia para que la universidad contacte.
- **Modificación mínima** de `components/explorar/OfferDetailModal.tsx` : `handleApplyClick` ahora abre el modal (antes hacía `alert`). Se conserva el evento `trackApplyAttempt` existente. No se tocó nada más del explorador.

5. Cumplimiento de restricciones

- ☒ Transacción atómica e idempotente (`clave_idempotencia` con índice único).
- ☒ **Nunca** transfiere sin consentimiento (`transferencias_universidad.consentimiento_id` NOT NULL + guarda de la RPC).
- ☒ Sin casillas preseleccionadas; textos desde `tipos_consentimiento` .
- ☒ Sin borrado físico; sin claves hardcodeadas.
- ☒ Reutiliza tablas reales; no crea paralelas.

6. Estado de verificación

- **Código escrito y coherente** con el diccionario de datos (nombres de columnas verificados en `BUSCOEDU_DATABASE_SCHEMA_FINAL.md`).
- **NO ejecutado**: sin credenciales Supabase no se pudo correr el RPC ni el flujo E2E. Queda **pre-parado**. La compilación se valida en Fase 6.
- La RPC es defensiva ante columnas opcionales (p. ej. `sede_id` con manejo de `undefined_column`).

7. Riesgos

Riesgo	Mitigación
Nombres de columnas divergentes en tablas hija	Se insertan solo columnas documentadas; probar al aplicar y ajustar.
No hay asesor ni <code>super_admin</code> activo	La oportunidad se crea igual (sin asignar); un <code>job/admin</code> puede asignarla luego.
<code>to_jsonb(v_oferta)</code> incluye columnas que cambien	Es un snapshot fiel al momento; es justamente el comportamiento deseado.